STAT 656 FINAL PROJECT REPORT

Non-Linear Regression for Environmental Data Modeling

Satvika Reddy Akati akati2004@tamu.edu **UIN: 137009580**

Hasini Kavuru hasini_03@tamu.edu **UIN : 837004778**

Thanusri Aenugula thanusriaenugula@tamu.edu **UIN : 437004805**

Summary: In this project, we wanted to see whether interpretable non-linear regression could predict hourly carbon monoxide concentrations from multisensor air-quality readings and weather conditions better than simpler linear methods. Using 9,358 hourly records collected in an Italian city and a cleaned analytic dataset of 9,357 rows, we compared cross-validated polynomial regression and smoothing splines. Our cubic polynomial model performed best, achieving test RMSE = 0.3988, MAE = 0.2641, and R^2 = 0.9236, showing that a carefully tuned classical regression model can predict environmental sensor data accurately while still staying easy to explain.

1. Project Overview and Problem Statement

Air pollution forecasting matters because even small changes in pollutant levels can affect public health and policy decisions. This project used the Air Quality Chemical Multisensor Device dataset to ask: can interpretable non-linear regression models use low-cost sensor readings and weather variables to accurately predict hourly carbon monoxide (CO) concentration in an urban setting.

We refined our scope during the project, selecting CO(GT) as the main response variable due to its superior availability versus NMHC(GT). This improved reproducibility, reduced instability from severe missing values, and aligned with course goals in regression, preprocessing, visualization, and model evaluation. The broader importance is that environmental agencies need models that are both accurate and transparent enough to support real monitoring decisions.

2. Data Acquisition, Sources, and Dataset Description

2.1 Data Source and Data Characteristics

In our project, we used the UCI Air Quality dataset, which is a multivariate time-series dataset containing hourly air-quality measurements collected from a chemical multisensor device in an Italian city. After loading the dataset and removing the formatting-related empty column, the raw dataset had a shape of 9,471 rows and 15 columns. We then cleaned the data, created additional time-based features, and prepared the final analytic dataset, which had 9,357 rows and 17 columns. The dataset includes pollutant measurements, sensor responses, meteorological variables as shown in Table 1, and missing values originally encoded as -200.

Since the dataset was provided as a semicolon-delimited CSV file with commas used as decimal separators, careful data loading was an important part of our workflow. We used pandas.read_csv with sep=';', decimal=',', and na_values=[-200] so that the file would be read correctly and the missing-value codes would be converted into standard NaN values. We did not need SQL or NoSQL queries for data collection.

```
df_raw = pd.read_csv(data_path,sep=";",decimal=",",na_values=[-200],engine="python")
```

Category	Examples	Role in pipeline	Notes
Temporal fields	Date, Time, engineered datetime, hour, day-of-week, month	Indexing, seasonality, behavior by hour/day	Date and time merged into a single timestamp

Category	Examples	Role in pipeline	Notes
Reference pollutant variables	CO(GT), NO2(GT), NOx(GT), NMHC(GT), C6H6(GT)	Target and contextual pollutant indicators	CO(GT) selected as final target
Sensor-response variables	PT08.S1(CO) through PT08.S5(O3)	Surrogate predictor from metal-oxide sensors	High cross-correlation, possible sensor drift
Meteorological variables	T, RH, AH	Environmental drivers and confounders	Useful for capturing curvature and seasonal effects

Table 1. Variable groups used in the final modeling pipeline

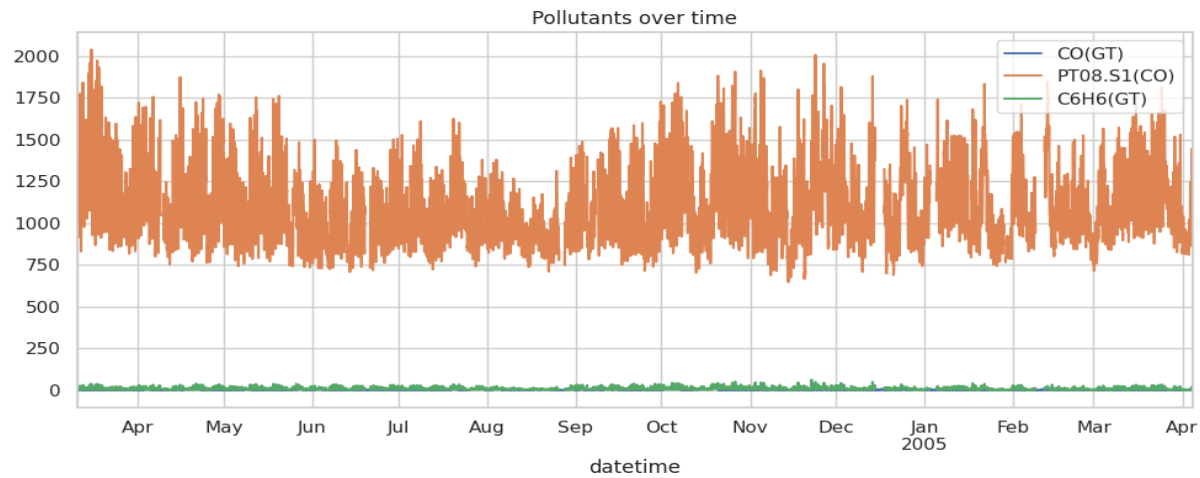

Figure 1. Pollutants over time

As shown in Figure 1, pollutant levels vary considerably over time and do not follow a simple constant pattern. This temporal fluctuation suggested that the relationship between predictors and CO concentration was likely dynamic and non-linear.

2.2 Limitations of the Dataset

- Univariate location: one Italian city only, limiting geographic generalizability.
- Severe missingness in NMHC(GT) (~90%) constrained target variable selection.
- Sensor drift in metal-oxide sensors may affect temporal stability of predictors.

3. Data Preprocessing and Cleaning

3.1 Initial Data Cleaning

In our project, data preprocessing was one of the most important steps because it directly affected the quality of our models. We first removed the completely empty trailing column that appeared. After that, we combined the Date and Time columns into a single datetime variable using the dataset's original DD/MM/YYYY and HH.MM.SS format. We also converted all measurement columns into numeric format so that any issues caused by the CSV's locale-specific formatting would not affect our analysis.

```
df["datetime"] = pd.to_datetime(
    df["Date"] + " " + df["Time"],
    format="%d/%m/%Y %H.%M.%S"
)

df["hour"] = df["datetime"].dt.hour
df["day_of_week"] = df["datetime"].dt.dayofweek
df["month"] = df["datetime"].dt.month
```

3.2 Missing Value Audit

We replaced all -200 values with missing values and carefully examined the missing-data pattern to guide both feature selection and target selection. From our missing-value analysis, we found that the missingness was not evenly distributed across variables. NMHC(GT) had 8,443 missing values, which was about 90.23% of the cleaned dataset, so we decided it was not suitable to use as the main target variable. In contrast, CO(GT), NO_2 (GT), and NOx(GT) had around 17.5% to 18.0% missing values, while many sensor-response and meteorological variables had only about 3.91% missingness.

3.3 Feature Selection and Engineering

We selected features that were both scientifically meaningful and available in a sufficient number of records. Our final modeling dataset used PT08.S1(CO), C6H6(GT), NO2(GT), T, RH, AH, hour, day-of-week, and month to predict CO(GT). For feature engineering, we mainly focused on capturing temporal structure instead of applying unnecessary transformations. We created variables such as hour, day-of-week, and month to represent daily patterns, weekly effects, and seasonal variation.

3.4 Outlier Handling and Data Validation

Before moving to modeling, we also reviewed summary statistics and plotted variable distributions to check for unusual values and skewness. We did not remove outliers blindly, because in an environmental dataset, extreme pollution readings can still be meaningful. Instead, we handled this carefully by relying on cross-validation and residual analysis to judge whether the model was learning the data in a stable and reliable way.

In Figure 2, the boxplots of pollutants reveal skewness, broad spread, and real outlying behavior, supporting a careful cleaning strategy that preserves environmental extremes instead of deleting them blindly. These distributions showed that the dataset contains both natural variability and extreme pollution events, which are meaningful for environmental modeling and should not automatically be treated as noise.

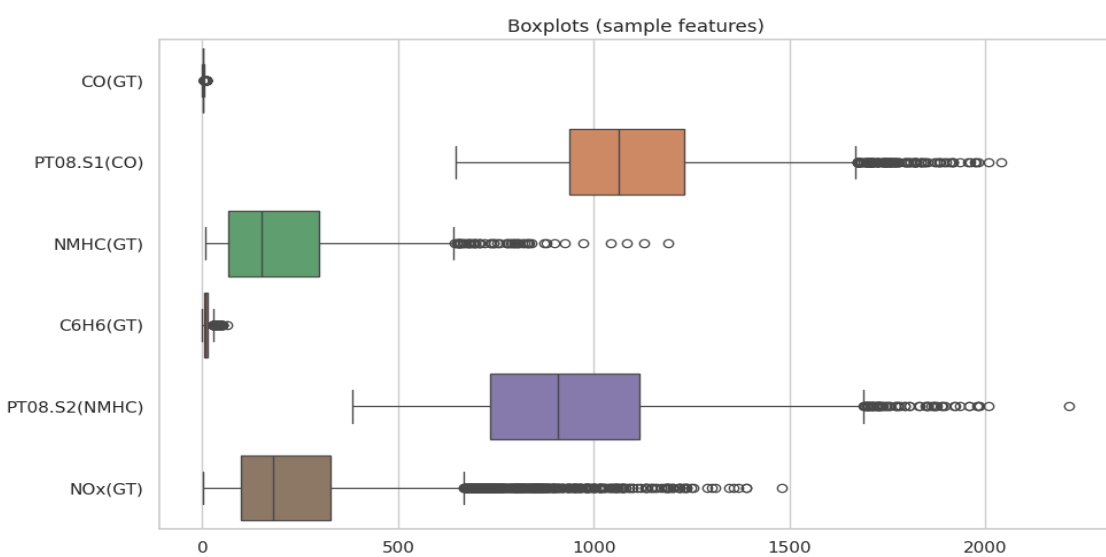

Figure 2. Boxplots for Pollutants

4. Exploratory Data Analysis and Statistical Analysis

The Spearman correlation heatmap in Figure 3 showed strong interrelationships among sensor channels, confirming useful predictive signal alongside nontrivial multicollinearity. A Spearman correlation of 0.73 between NO_2 and CO was found.

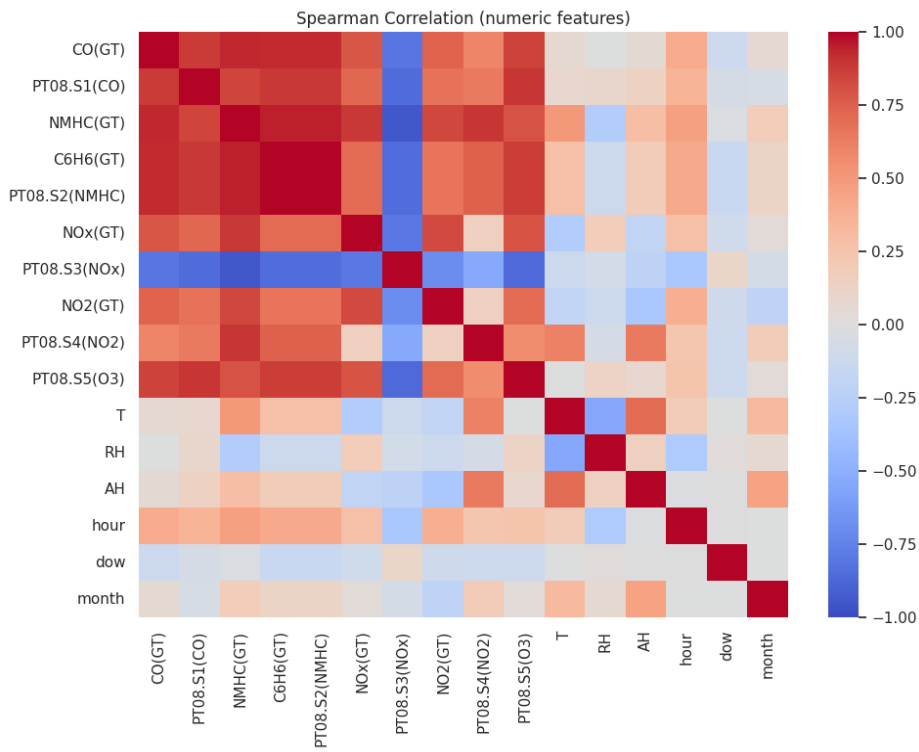

Figure 3. Spearman correlation heatmap for numerical features

Our exploratory data analysis showed that the problem we were studying is clearly non-linear. We observed that pollutant levels changed over time, varied across different hours of the day, and showed curved relationships with both weather variables and other pollutants. The summary statistics also showed noticeable variation across the measurements, and the Spearman correlation heatmap helped us see that several sensor channels moved together, which is expected in a multisensor environmental monitoring dataset.

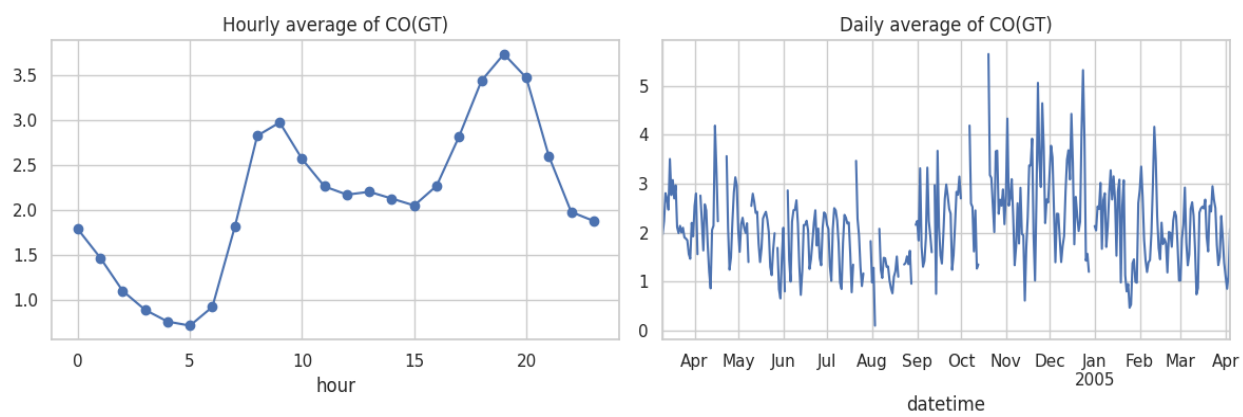

Figure 4. Hourly and daily averages for CO(GT)

When we looked more closely at the hourly CO pattern, we found that the mean CO concentration ranged from about 0.71 to 3.73 across the day. This suggested a strong daily pattern, likely influenced by traffic activity and atmospheric conditions. In our notebook, we also calculated rank-based correlations and found a strong positive

Spearman correlation of 0.73 between NO2 and CO. On the other hand, temperature and humidity showed weaker monotonic correlations with CO. However, we did not treat those weaker correlations as unimportant. When we examined scatterplots with polynomial trend lines, we could clearly see curved relationships, especially for temperature, which further supported our decision to use non-linear modeling methods. As shown in Figure 4, the left panel shows a strong hourly pattern, while the right panel shows substantial day-to-day variation. Together, these trends justify our temporal feature engineering choices.

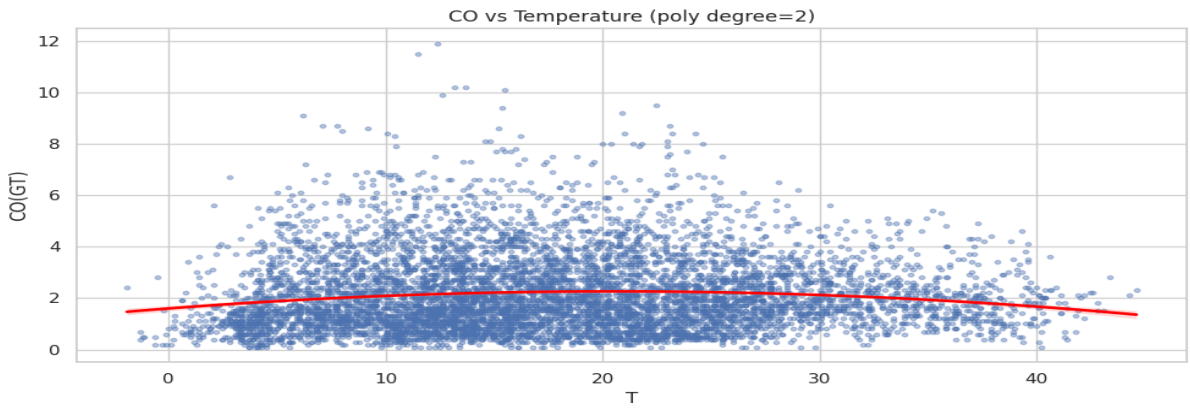

Figure 5. CO versus temperature with a degree-2 trend line

As shown in Figure 5, the curved trend makes it clear that a simple linear model would miss important patterns in the data. Based on our EDA results, we felt that a purely linear approach would likely underfit because the relationships between the predictors and the response were curved, seasonal, and partly dependent on one another. For this reason, we focused on non-linear methods such as polynomial feature expansion and spline smoothing, while still keeping the models interpretable instead of moving to a completely black-box approach.

Our distributional analysis also helped us make modeling decisions more carefully. We found that several pollutant variables were right-skewed, but based on the checks we performed, no feature clearly required a mandatory log1p transformation under our screening approach. Because of this, we used residual diagnostics as the final step to judge whether the selected model was handling systematic prediction error properly and whether the fitted model was appropriate for the data.

5. Machine Learning Models

5.1 Polynomial Regression And Smoothing Spline

A major part of our machine-learning work was comparing polynomial regression and smoothing spline regression using the same train/test setup. After keeping only the rows with complete values for our final feature set, we split the data into 6,139 training records and 1,535 test records. We built polynomial regression as a pipeline that combined feature expansion with linear regression, and then we tested polynomial degrees from 1 to 8. From these experiments, we saw that degrees above 4 started overfitting very quickly. The training RMSE became extremely small, but the test RMSE increased sharply, with degree 7 going above 6,700. This clearly showed us that cross-validation was necessary for choosing the right model complexity.

Model	CV RMSE	CV MAE	CV R^2	Test RMSE	Test MAE	Test R^2
Polynomial regression (degree 3)	0.4273	0.2760	0.9137	0.3988	0.2641	0.9236
Smoothing spline + Ridge	0.4944	0.3099	0.8842	0.4899	0.3041	0.8848

Table 2. Comparison of the two classical models

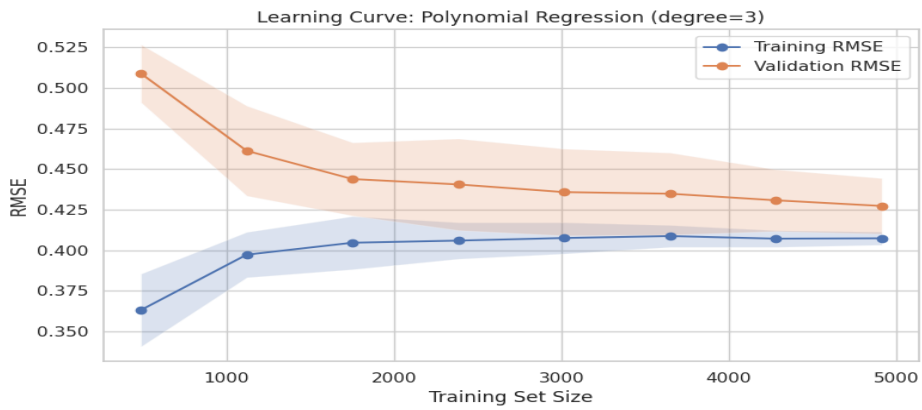

Figure 6. Learning curve for the polynomial model

Using five-fold cross-validation, we found that degree 3 was the best polynomial model, with a mean CV RMSE of 0.4218 and fairly stable performance across folds. For the spline-based model, we used SplineTransformer together with Ridge regularization. We then applied grid search, which selected 6 knots, degree 3 splines, and an alpha value of 0.1 as the best combination, giving a CV RMSE of 0.4944. As shown in Table 2, both models were interpretable and followed a clear modeling process, but the polynomial model performed better overall and generalized more effectively on our dataset. In addition, Figure 6 shows that the training and validation RMSE curves moved closer as the training size increased, which suggests that the selected polynomial model generalized well instead of simply memorizing the training data.

6. Deep Learning Model

6.1 PyTorch MLP Architecture & Training

To complement the classical models, we implemented a PyTorch-based multilayer perceptron (MLP) as a deep learning baseline for tabular regression. The model used standardized numerical input features and consisted of three fully connected hidden layers with 128, 64, and 32 neurons, followed by ReLU activation functions. Dropout with a rate of 0.1 was applied between hidden layers to reduce overfitting.

```
class MLP(nn.Module):
    def __init__(self, input_dim):
        super(MLP, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 128),nn.ReLU(),nn.Dropout(0.1),
            nn.Linear(128, 64),nn.ReLU(),nn.Dropout(0.1),
            nn.Linear(64, 32),nn.ReLU(),
            nn.Linear(32, 1)
        )
```

For training, we used Mean Squared Error (MSE) as the loss function and the Adam optimizer with a learning rate of 0.0005 and weight decay of 1e-4. The model was trained for up to 150 epochs, with early stopping based on validation loss so that training would stop once performance no longer improved. We also tracked training and validation loss across epochs to evaluate convergence and learning behavior.

To ensure fair comparison with the classical models, we evaluated the PyTorch model using the same 5-fold cross-validation strategy. As shown in Table 3, the PyTorch MLP achieved a CV RMSE of 0.5890, CV MAE of 0.3926, and CV R^2 of 0.8356. On the held-out test set, it achieved RMSE = 0.5821, MAE = 0.3920, and R^2 = 0.8373. Figure 7 shows the cubic polynomial model is consistently stronger across the main predictive metrics.

Although the neural network was able to capture non-linear relationships in the data, it did not outperform the cubic polynomial regression model. From a methodological standpoint, the PyTorch model serves as a flexible benchmark rather than the primary solution. These results show that, for this tabular environmental dataset, interpretable non-linear regression provided a better balance of accuracy, stability, and explainability. This comparison also highlights that more complex models do not always produce better results, especially when the dataset size is moderate and the underlying relationships can already be captured effectively by structured statistical models.

Model	CV RMSE	CV MAE	CV R^2	Test RMSE	Test MAE	Test R^2
PyTorch MLP	0.5890	0.3926	0.8356	0.5821	0.3920	0.8373

Table 3. Performance metrics of PyTorch MLP.

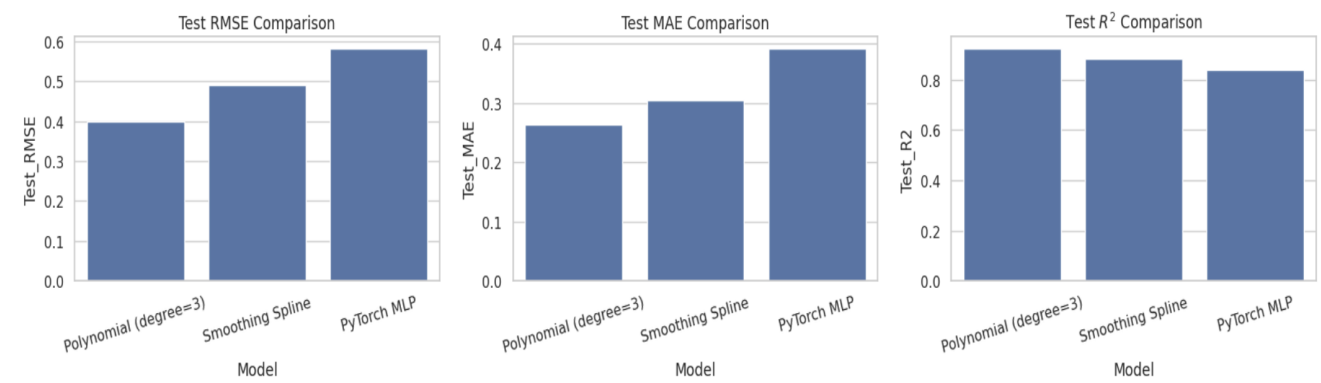

Figure 7. Visual comparison of RMSE, MAE, and R^2

7. Results, Model Evaluation, and Key Insights

7.1 Final Model Performance

The final selected model was polynomial regression of degree 3. It achieved Train RMSE = 0.4073, Test RMSE = 0.3988, Train MAE = 0.2660, Test MAE = 0.2641, Train R^2 = 0.9217, and Test R^2 = 0.9236. The close match between training and test performance suggests that the model generalized well and did not show harmful overfitting at the selected degree.

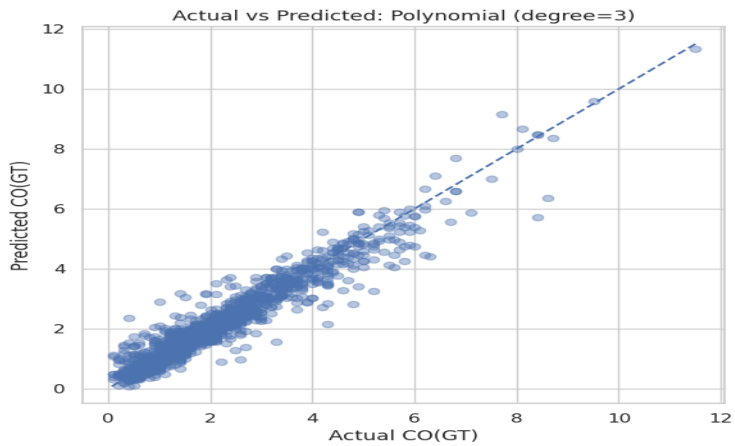

Figure 8. Actual versus predicted CO(GT)

The smoothing spline model also performed reasonably well, but it remained weaker than the polynomial model on both error metrics and explained variance. We also evaluated a PyTorch MLP as a deep learning baseline. Although it was able to learn non-linear relationships in the data, its performance was lower than both classical models, with CV RMSE = 0.5890, CV MAE = 0.3926, CV R^2 = 0.8356, Test RMSE = 0.5821, Test MAE = 0.3920, and Test R^2 = 0.8373. Overall, these results showed that the cubic polynomial regression model provided the best balance of predictive accuracy, stability, and interpretability for our dataset. As shown in Figure 8, the actual and predicted CO(GT) values closely follow the 45-degree reference line, confirming that the model predicted the target accurately across most of the observed range.

7.2 Feature Importance

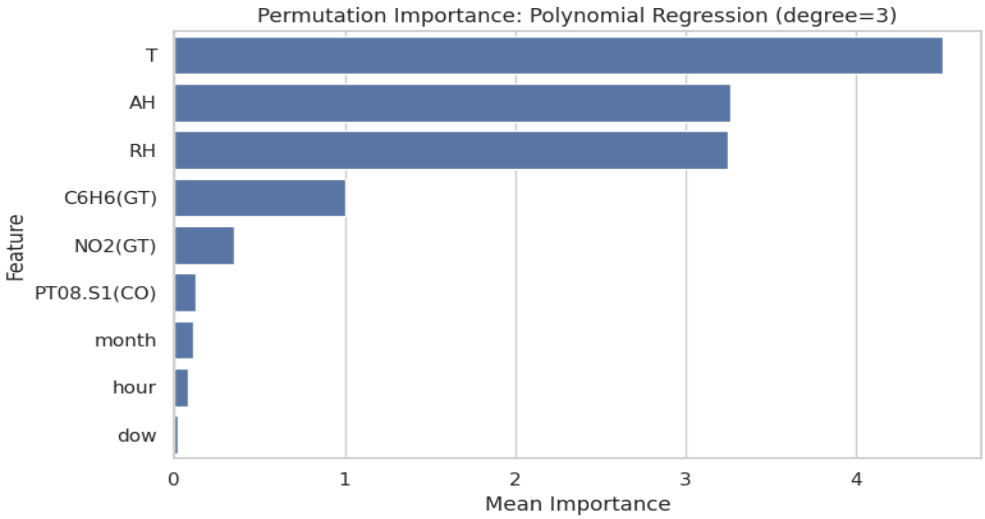

Figure 9. Permutation importance for the final model

Permutation importance in Figure 9 helped us understand the model beyond just accuracy. Temperature was the most important feature, followed by absolute humidity and relative humidity, showing that weather conditions played a major role in predicting CO concentration. Benzene and NO2 also contributed meaningfully, which matches the expected relationship among urban pollutants. In addition, the residual plots were centered close to zero without major patterns, suggesting that the final model fit the data well. Overall, our results showed that we did not have to choose between interpretability and accuracy, because the cubic polynomial model provided both.

7.3 Key Insights

- Cross-validation is critical: higher-degree polynomials produced near-zero training error but collapsed on the test set.
- Interpretability does not require sacrificing accuracy: cubic polynomial achieved $R^2 = 0.9236$ while remaining fully interpretable.
- Deep learning does not always win: the PyTorch MLP underperformed both classical models on this moderate-sized tabular dataset.
- Weather dominates CO prediction: temperature and humidity were the top features; sensor proxies provided secondary signal.

8. Challenges, Limitations, Societal Impact, and Reproducibility

8.1 Technical Challenges

- Data parsing: semicolon delimiters, decimal commas, trailing empty column, and −200-encoded missing values required careful handling.
- Target selection: severe missingness in NMHC(GT) forced switching to CO(GT), requiring pipeline re-alignment.
- Overfitting risk: higher-degree polynomials required disciplined cross-validation to identify safe complexity.

8.2 Limitations

The dataset covers one city and one time period, limiting generalizability. Listwise deletion may have discarded useful information. The model is predictive, not causal; feature importance should not be interpreted as physical causation. The PyTorch baseline was not tuned as extensively as classical models.

8.3 Societal and Ethical Implications

Better CO prediction can support public-health alerts, urban planning, and environmental compliance. Inaccurate predictions or sensor drift could mislead decision-makers, making transparency and residual diagnostics essential. Privacy concerns are minimal; environmental justice remains relevant as pollution burdens fall unevenly across communities.

9. Conclusion and Future Work

9.1 Conclusion

This project built an end-to-end pipeline from raw UCI CSV ingestion through cleaning, EDA, feature engineering, non-linear model training, and final evaluation. The cubic polynomial regression model (degree 3) emerged as the best solution: test RMSE = 0.3988, MAE = 0.2641, $R^2 = 0.9236$, while remaining fully interpretable. Results demonstrate that interpretable non-linear regression can perform effectively on structured environmental datasets without sacrificing accuracy for transparency.

9.2 Future Work

Future directions include further MLP hyperparameter tuning, time-aware validation strategies, imputation methods instead of listwise deletion, sensor drift correction, and extending to other pollutants such as NO_2 or additional air-quality datasets.

9.3 Reproducibility

All code is in the project Jupyter notebook and reproducible by running cells sequentially. Required libraries: Python, PyTorch, pandas, numpy, matplotlib, scikit-learn. Fixed random state (random_state=42) is used throughout. All code is contained in STAT656 Group 9 Code.ipynb. The Github Link for the code is shared.

10. Team Contributions and Collaboration

Team member	Primary contributions	Collaboration evidence
Satvika Reddy Akati	Dataset inspection, missing-value audit, cleaning logic, time-feature construction, and early EDA plots and report writing	Worked on raw-data understanding and shared preprocessing decisions during weekly milestones.
Hasini Kavuru	Polynomial regression experiments, cross-validation, error analysis, learning-curve interpretation, and report drafting.	Compared model degrees, summarized overfitting behavior, and integrated results into the written narrative.
Thanusri Aenugula	Spline-model, Deep learning model development, final model comparison, permutation importance, conclusion/future-work synthesis, and report writing.	Consolidated exported tables/figures and ensured the final report matched the proposal-to-final-project progression.

Table 4. Contribution summary

References

UCI Machine Learning Repository. Air Quality dataset. https://doi.org/10.24432/CS59K5F

GitHub repository: Air Quality Project. https://github.com/satvikareddyakati/airquality